

Module 3 : Introduction to OOPS Programming

1. Introduction to C++

Q1. What are the key differences between Procedural Programming and Object-Oriented Programming (OOP)?

→**Procedural Programming:**

1. Programming Approach

Procedural Programming uses a linear, step-by-step method to solve problems. The program is organized into separate functions.

2. Program Focus

The main emphasis is on writing procedures that perform tasks, not on protecting data.

3. Data Security

Data protection is weak because functions can freely access and modify shared data.

4. Code Reusability

Reusability is lower since functions and data are not tightly connected.

5. Program Complexity

As the program size increases, it becomes harder to control and manage.

6. Program Maintenance

Updating or modifying large programs can be complicated and time-consuming.

7. Real-World Representation

It does not naturally model real-life objects and relationships.

Object-Oriented Programming (OOP):

1. Programming Approach

Object-Oriented Programming organizes programs around classes and objects that bundle data with related functions.

2. Program Focus

The primary focus is on data and how objects interact with each other.

3. Data Security

It provides strong data protection through concepts like encapsulation and access control.

4. Code Reusability

Programs are highly reusable because of features like inheritance and polymorphism.

5. Program Complexity

It is well-suited for handling large-scale and complex applications.

6. Program Maintenance

Programs are easier to update, debug, and expand due to structured design.

7. Real-World Representation

It closely simulates real-world objects and their interactions.

Basis	Procedural Programming	Object-Oriented Programming (OOP)
Approach	Top-down approach	Bottom-up approach
Focus	Focuses on functions/procedures	Focuses on objects and classes
Program Structure	Divided into functions	Divided into objects
Data Security	Data is less secure (global data can be accessed easily)	Data is more secure (uses encapsulation)
Reusability	Limited reusability	High reusability (inheritance)
Real-world Modeling	Not based on real-world concepts	Based on real-world entities
Examples	C, Pascal	C++, Java, Python

Q2. List and explain the main advantages of OOP over POP.



Main Advantages of OOP over POP

1. Better Data Security

OOP provides data hiding through **encapsulation**. Data cannot be accessed directly from outside the class, which makes programs more secure compared to POP.

2. Code Reusability

OOP supports **inheritance**, which allows one class to reuse the properties and methods of another class. This reduces code duplication.

3. Easier Maintenance

Programs developed using OOP are easier to update and modify because changes in one class do not affect the entire program.

4. Handles Large Programs Efficiently

OOP is suitable for developing large and complex applications as it divides the program into small, manageable objects.

5. Real-World Modeling

OOP closely represents real-world entities using objects, making programs easier to design and understand.

6. Flexibility through Polymorphism

With polymorphism, the same function can perform different tasks, increasing flexibility and scalability of programs.

Q3. Explain the steps involved in setting up a C++ development environment.



1. Install a C++ Compiler

A compiler is required to convert C++ code into machine language.

Examples include **GCC (MinGW)** or the compiler included in **Visual Studio**.

2. Choose and Install an IDE or Code Editor

An IDE (Integrated Development Environment) makes coding easier by providing editing, compiling, and debugging tools.

Examples: Code::Blocks, Dev-C++, Visual Studio, or VS Code.

3. Configure the Compiler

After installation, link the compiler with the IDE (if not automatically configured).

This ensures the IDE can compile and run C++ programs.

4. Create a New C++ File

Create a new file and save it with the .cpp extension (example: program.cpp).

5. Write the C++ Program

Type your C++ code in the editor.

6. Compile the Program

Compile the program to check for errors.

If there are no errors, the compiler generates an executable file.

7. Run the Program

Execute the compiled program to see the output.

Q4. What are the main input/output operations in C++? Provide examples.

→

i. cout (Output Operation)

- Used to display output on the screen.
- It uses the insertion operator <<.

Example:

```
#include <iostream>

using namespace std;

int main() {
    cout << "Welcome to C++";
    return 0;
}
```

ii. cin (Input Operation)

- Used to take input from the user.
- It uses the extraction operator >>.

Example:

```
#include <iostream>

using namespace std;

int main() {
    int num;
    cin >> num;
    cout << "You entered: " << num;
}
```

2. Variables, Data Types, and Operators

Q1. What are the different data types available in C++? Explain with examples.



Data types in C++ define the type of data a variable can store.

1 Basic (Primitive) Data Types

These are fundamental data types provided by C++.

Data Type	Description	Example
int	Stores whole numbers	int a = 10;
float	Stores decimal numbers	float b = 5.5;
double	Stores large decimal numbers	double c = 10.12345;
char	Stores single character	char grade = 'A';
bool	Stores true or false	bool flag = true;

2 Derived Data Types

These are derived from basic data types.

Data Type	Description	Example
Array	Collection of same type elements	int arr[5];
Pointer	Stores address of variable	int *ptr;
Function	Block of code performing task	int add(int a, int b);

3 User-Defined Data Types

These are created by the user.

Data Type	Description	Example
struct	Groups different data types	struct Student { int roll; };
class	Blueprint for objects	class Car { };
union	Shares memory among members	union Data { int x; float y; };
enum	Named constants	enum Day {Mon, Tue};

Q2. Explain the difference between implicit and explicit type conversion in C++.

→ Type conversion means changing one data type into another data type.

1 Implicit Type Conversion (Automatic)

Implicit type conversion is performed automatically by the compiler when different data types are used in an expression.

Features:

- Done automatically
- No special syntax required
- Usually converts smaller type to larger type (e.g., int → float)

Example:

```
#include <iostream>
using namespace std;
int main() {
    int a = 10;
    float b = a; // int automatically converted to float
    cout << b;
}
```

2 Explicit Type Conversion (Type Casting)

Explicit type conversion is done manually by the programmer using casting.

Features:

- Done by programmer
- Uses casting operator like (type) or static_cast
- May cause data loss (e.g., float → int)

Example:

```
#include <iostream>
using namespace std;
int main() {
    float x = 9.8;
    int y = (int)x; // explicit conversion
    cout << y;
    return 0;
}
```


Q3. What are the different types of operators in C++? Provide examples of each.

→ Operators are special symbols used to perform operations on variables and values.

1 Arithmetic Operators

Used to perform mathematical calculations.

Operator	Meaning	Example
+	Addition	a + b
-	Subtraction	a - b
*	Multiplication	a * b
/	Division	a / b
%	Modulus	a % b

Example:

```
int a = 10, b = 5;
```

```
cout << a + b; // 15
```

2 Relational (Comparison) Operators

Used to compare two values.

Operator	Meaning	Example
==	Equal to	a == b
!=	Not equal	a != b
>	Greater than	a > b
<	Less than	a < b
>=	Greater than or equal	
<=	Less than or equal	

Example:

```
cout << (a > b); // 1 (true)
```

3 Logical Operators

Used to combine conditions.

Operator	Meaning
&&	Logical AND
,	
!	Logical NOT

Example:

```
if(a > 5 && b < 10)  
    cout << "True";
```

4 Assignment Operators

Used to assign values to variables.

Operator	Example
=	a = 10
+=	a += 5
-=	a -= 5
*=	a *= 2
/=	a /= 2

5 Increment and Decrement Operators

Operator	Meaning
++	Increase by 1
--	Decrease by 1

Example:

```
int x = 5;
```

```
x++; // x becomes 6
```

6 Bitwise Operators

Used to perform operations on bits.

Operator	Meaning
&	AND
,	,
^	XOR
~	NOT
<<	Left Shift
>>	Right Shift

7 Ternary (Conditional) Operator

Syntax:

```
condition ? expression1 : expression2;
```

Example:

```
int result = (a > b) ? a : b;
```

8 Special Operators

- sizeof
- & (address of)
- * (pointer)
- :: (scope resolution)

Q4. Explain the purpose and use of constants and literals in C++.

→

1 Constants

A **constant** is a fixed value that cannot be changed during program execution.

Purpose of Constants:

- To store values that should not be modified.
- To make programs safer and more reliable.
- To improve code readability.
- To avoid accidental changes in important values.

How to Declare Constant:

Using const keyword:

```
const int age = 20;
```

2 Literals

A **literal** is a fixed value that is directly written in the program.

Literals are actual values assigned to variables.

Types of Literals in C++:

Type	Example	Description
Integer	10, -5	Whole numbers
Floating	3.14, 2.5	Decimal numbers
Character	'A', 'b'	Single character
String	"Hello"	Group of characters
Boolean	true, false	Logical values

Difference Between Constants and Literals

Basis	Constants	Literals
Definition	Fixed value stored in a variable	Fixed value written directly in program
Keyword	Uses const keyword	No keyword required
Name	Has a variable name	No name
Change	Cannot be changed	Value itself is fixed
Example	const int a = 10;	10, 3.14, 'A', "Hello"

3. Control Flow Statements

Q1. What are conditional statements in C++? Explain the if-else and switch statements.



Conditional statements are used to make decisions in a program.

They execute different blocks of code based on whether a condition is **true or false**.

In C++, the main conditional statements are:

- if
- if-else
- else if
- switch

1 if-else Statement

Used to execute one block of code if the condition is true and another block if it is false.

Syntax:

```
if (condition) {  
    // code if condition is true  
}  
else {  
    // code if condition is false  
}
```

2 switch Statement

Used to select one block of code from multiple options based on the value of a variable.

Syntax:

```
switch(expression) {  
    case value1:  
        // code  
        break;  
    case value2:  
        // code  
        break;  
    default:  
        // default code  
}
```

Q2. What is the difference between for, while, and do-while loops in C++?

→

Basis	for Loop	while Loop	do-while Loop
Condition Check	Before loop starts	Before loop starts	After loop execution
Minimum Execution	May not run	May not run	Runs at least once
Best Used When	Iterations known	Iterations unknown	At least one execution needed
Syntax Structure	Initialization inside	Initialization outside	Condition at end

Q3. How are break and continue statements used in loops? Provide examples.



1 break Statement

Purpose:

The break statement is used to immediately terminate the loop and exit from it.

When break is executed:

- The loop stops.
- Control moves to the next statement after the loop.

Example:

```
#include <iostream>
using namespace std;
```

```
int main() {
    for(int i = 1; i <= 10; i++) {
        if(i == 5) {
            break; // loop stops when i becomes 5
        }
        cout << i << " ";
    }
    return 0;
}
```

Output:

1 2 3 4

2 continue Statement

Purpose:

The continue statement skips the current iteration and moves to the next iteration of the loop.

When continue is executed:

- Remaining code inside the loop is skipped.
- Loop continues with next iteration.

Example:

```
#include <iostream>
using namespace std;
```

```
int main() {
    for(int i = 1; i <= 5; i++) {
        if(i == 3) {
            continue; // skip number 3
        }
        cout << i << " ";
    }
    return 0;
}
```

Output:

1 2 4 5

Q4. Explain nested control structures with an example.



A **nested control structure** means placing one control statement inside another control statement.

When a loop or condition is written inside another loop or condition, it is called **nested control structure**.

Types of Nesting

1. Nested if statement
2. Nested loops (for, while, do-while)
3. if inside loop or loop inside if

Example **1** : Nested if Statement

```
#include <iostream>
```

```
using namespace std;
```

```
int main() {
```

```
    int num = 10;
```

```
    if(num > 0) {        // Outer if
```

```
        if(num % 2 == 0) { // Inner if
```

```
            cout << "Positive and Even";
```

```
        }
```

```
    }
```

```
    return 0;
```

```
}
```

Explanation:

- First if checks if number is positive.
- Second if checks if number is even.
- Since 10 is positive and even → output is displayed.

Example 2 : Nested for Loop

```
#include <iostream>
```

```
using namespace std;
```

```
int main() {
```

```
    for(int i = 1; i <= 3; i++) {    // Outer loop
```

```
        for(int j = 1; j <= 2; j++) { // Inner loop
```

```
            cout << i << " " << j << endl;
```

```
        }
```

```
    }
```

```
    return 0;
```

```
}
```

Explanation:

- Outer loop runs 3 times.
- Inner loop runs 2 times for each outer loop.
- Total executions = $3 \times 2 = 6$ times.

4. Functions and Scope

Q1. What is a function in C++? Explain the concept of function declaration, definition, and calling.



A **function** is a block of code that performs a specific task. It helps in breaking a large program into smaller, manageable parts.

Functions improve:

- Code reusability
- Program readability
- Easy debugging

Three Main Parts of a Function

1. Function Declaration
2. Function Definition
3. Function Calling

1 Function Declaration (Prototype)

It tells the compiler about:

- Function name
- Return type
- Parameters

It is written before main().

Syntax:

```
return_type function_name(parameter_list);
```

Example:

```
int add(int, int);
```

This tells the compiler that a function named add exists.

2 Function Definition

It contains the actual body of the function (logic).

Syntax:

```
return_type function_name(parameters) {  
    // statements  
}
```

Example:

```
int add(int a, int b) {  
    return a + b;  
}
```

This defines what the function actually does.

3 Function Calling

Calling a function means using it inside main() or another function.

Example:

```
int result = add(5, 3);
```

Here, add(5,3) calls the function and stores result.

Complete Program Example

```
#include <iostream>

using namespace std;

// Function Declaration
int add(int, int);

int main() {

    int sum = add(10, 5); // Function Calling
    cout << "Sum = " << sum;

    return 0;
}

// Function Definition
int add(int a, int b) {
    return a + b;
}
```

Output:

Sum = 15

Q2. What is the scope of variables in C++? Differentiate between local and global scope.

→

The **scope of a variable** refers to the region of a program where the variable can be accessed or used.

In C++, scope mainly depends on **where the variable is declared**.

There are two main types:

1. Local Scope
2. Global Scope

1 Local Scope

A variable declared inside a function or block { } is called a **local variable**.

Features:

- Accessible only inside that function/block.
- Cannot be used outside it.
- Created when function starts.
- Destroyed when function ends.

Example:

```
#include <iostream>
using namespace std;
```

```
int main() {
    int x = 10; // Local variable
    cout << x;
    return 0;
}
```

x cannot be used outside main().

2 Global Scope

A variable declared outside all functions is called a **global variable**.

Features:

- Accessible throughout the program.
- Can be used in multiple functions.
- Exists for the entire program execution.

Example:

```
#include <iostream>

using namespace std;
```

```
int x = 20; // Global variable
```

```
int main() {
    cout << x;
    return 0;
}
```

x can be used in any function.

Difference Between Local and Global Scope

Basis	Local Variable	Global Variable
Declaration	Inside function/block	Outside all functions
Accessibility	Only within that block	Whole program
Lifetime	Till function ends	Till program ends
Memory	Created when function runs	Created at program start

Q3. Explain recursion in C++ with an example.



Recursion is a process in which a function calls itself to solve a problem.

A recursive function must have:

1. Base case → Stops the recursion
2. Recursive case → Function calls itself

Without a base case, the function will run infinitely.

C++ Program:

```
#include <iostream>
```

```
using namespace std;
```

```
int factorial(int n) {  
    if(n == 0)        // Base case  
        return 1;  
    else  
        return n * factorial(n - 1); // Recursive call  
}
```

```
int main() {  
    int num = 5;  
    cout << "Factorial = " << factorial(num);  
    return 0;  
}
```

Output:

Factorial = 120

How It Works (Step-by-Step)

For factorial(5):

- $5 \times \text{factorial}(4)$
- $4 \times \text{factorial}(3)$
- $3 \times \text{factorial}(2)$
- $2 \times \text{factorial}(1)$
- $1 \times \text{factorial}(0)$
- $\text{factorial}(0) = 1$ (Base case)

Then values return back:

$$5 \times 4 \times 3 \times 2 \times 1 = 120$$

Advantages of Recursion

- Makes code shorter and cleaner
- Useful for problems like:
 - Factorial
 - Fibonacci
 - Tree traversal

Disadvantages

- Uses more memory (function call stack)
- Slower than loops in some cases

Q4. What are function prototypes in C++? Why are they used?



A **function prototype** is a declaration of a function that tells the compiler:

- Function name
- Return type
- Number of parameters
- Type of parameters

It does **not** contain the function body.

Syntax:

```
return_type function_name(parameter_list);
```

Example:

```
int add(int, int);
```

This tells the compiler that:

- There is a function named add
- It returns an int
- It takes two int parameters

Why Are Function Prototypes Used?

1 To Inform the Compiler

C++ requires functions to be declared before they are used.

If the function is defined after main(), a prototype is necessary.

2 To Enable Function Calling Before Definition

It allows calling a function in main() even if its definition appears later in the program.

3 To Perform Type Checking

The compiler checks:

- Correct number of arguments
- Correct data types

This helps prevent errors.

Example Program

```
#include <iostream>
```

```
using namespace std;
```

```
// Function Prototype
```

```
int multiply(int, int);
```

```
int main() {
```

```
    int result = multiply(4, 5); // Function call
```

```
    cout << "Result = " << result;
```

```
    return 0;
```

```
}
```

```
// Function Definition
```

```
int multiply(int a, int b) {
```

```
    return a * b;
```

```
}
```

Important Points

- Prototype ends with a semicolon (;)
- Parameter names are optional in prototype
- Must match the function definition exactly

5. Arrays and Strings

Q1. What are arrays in C++? Explain the difference between single-dimensional and multi-dimensional arrays.



An **array** is a collection of elements of the same data type stored in contiguous memory locations.

All elements are accessed using an index number.

Indexing in C++ starts from **0**.

Syntax:

```
data_type array_name[size];
```

Example:

```
int arr[5] = {10, 20, 30, 40, 50};
```

Types of Arrays

1. Single-Dimensional Array
2. Multi-Dimensional Array

1 Single-Dimensional Array

It is a simple list of elements stored in a single row.

Example:

```
#include <iostream>
```

```
using namespace std;
```

```
int main() {  
    int marks[3] = {85, 90, 78};  
  
    for(int i = 0; i < 3; i++) {  
        cout << marks[i] << " ";  
    }  
  
    return 0;  
}
```

Output:

85 90 78

Uses only one index: marks[i]

2 Multi-Dimensional Array

An array with more than one dimension.
Most common is **2D array** (rows and columns).

Syntax:

```
data_type array_name[row][column];
```

Example:

```
#include <iostream>  
  
using namespace std;  
  
int main() {  
    int matrix[2][2] = {{1, 2}, {3, 4}};
```

```

for(int i = 0; i < 2; i++) {
    for(int j = 0; j < 2; j++) {
        cout << matrix[i][j] << " ";
    }
    cout << endl;
}

return 0;
}

```

Output:

1 2

3 4

Uses two indexes: matrix[i][j]

Difference Between Single and Multi-Dimensional Arrays

Basis	Single-Dimensional	Multi-Dimensional
Index Used	One index	Two or more indexes
Structure	List (Row)	Table (Rows & Columns)
Example	marks[5]	matrix[2][3]
Access Method	arr[i]	arr[i][j]

Q2. Explain string handling in C++ with examples.



In C++, strings can be handled in two ways:

1. **Using Character Arrays (C-style strings)**
2. **Using string Class (Standard Library)**

1 C-Style Strings (Character Arrays)

These are arrays of characters ending with a null character `\0`.

Example:

```
#include <iostream>

using namespace std;
```

```
int main() {
    char name[20] = "Hello";
    cout << name;
    return 0;
}
```

Common Functions (<cstring> library):

- `strlen()` → Finds length
- `strcpy()` → Copies string
- `strcat()` → Concatenates strings
- `strcmp()` → Compares strings

Example:

```
#include <iostream>
#include <cstring>
using namespace std;

int main() {
    char str1[20] = "Hello";
    char str2[20] = "World";

    strcat(str1, str2);
    cout << str1;

    return 0;
}
```

Output:

HelloWorld

2 String Class (#include <string>)

C++ provides a built-in string class which is easier and safer to use.

Example:

```
#include <iostream>
#include <string>
using namespace std;
```

```
int main() {  
    string name = "Hello";  
    cout << name;  
    return 0;  
}
```

Common String Operations

1 Concatenation

```
string a = "Hello ";  
string b = "World";  
cout << a + b;
```

Output:

Hello World

2 Length of String

```
string text = "Programming";  
cout << text.length();
```

3 Access Characters

```
string word = "CPlusPlus";  
cout << word[0]; // C
```

4 Taking Input with Spaces

```
string fullName;  
getline(cin, fullName);
```

Q3. How are arrays initialized in C++? Provide examples of both 1D and 2D arrays.

→

Array initialization means assigning values to array elements at the time of declaration.

1 Initialization of 1D Array (Single-Dimensional Array)

A 1D array stores elements in a single row.

◆ Syntax:

```
datatype array_name[size] = {values};
```

◆ Example 1: Full Initialization

```
#include <iostream>
```

```
using namespace std;
```

```
int main() {
```

```
    int arr[5] = {10, 20, 30, 40, 50};
```

```
    for(int i = 0; i < 5; i++) {
```

```
        cout << arr[i] << " ";
```

```
    }
```

```
    return 0;
```

```
}
```

◆ Example 2: Size Automatically Determined

```
int arr[] = {1, 2, 3, 4};
```

Here, the compiler automatically sets size to 4.

◆ Example 3: Partial Initialization

```
int arr[5] = {10, 20};
```

Remaining elements will be initialized to 0.

2 Initialization of 2D Array (Multi-Dimensional Array)

A 2D array stores data in rows and columns (like a matrix).

◆ Syntax:

```
datatype array_name[rows][columns] = {{values}};
```

◆ Example 1: 2×3 Matrix Initialization

```
#include <iostream>
```

```
using namespace std;
```

```
int main() {
```

```
    int matrix[2][3] = {
```

```
        {1, 2, 3},
```

```
        {4, 5, 6}
```

```
    };
```

```
    for(int i = 0; i < 2; i++) {
```

```
        for(int j = 0; j < 3; j++) {
```

```
            cout << matrix[i][j] << " ";
```

```
        }
```

```
        cout << endl;
```

```
    }
```

```
    return 0;  
}
```

◆ Example 2: Without Inner Braces

```
int matrix[2][2] = {1, 2, 3, 4};
```

This is stored as:

1 2

3 4

Difference Between 1D and 2D Array Initialization

Feature	1D Array	2D Array
Structure	Single row	Rows and columns
Declaration	int a[5];	int a[2][3];
Initialization	{1,2,3}	{{1,2},{3,4}}
Access	a[i]	a[i][j]

Q4. Explain string operations and functions in C++.

→

In C++, string operations can be performed using:

1. **C-style strings** (char[] with <cstring>)
2. **string class** (#include <string>) – recommended method

1 String Operations Using string Class

Include:

```
#include <string>
```

◆ 1. Concatenation (Joining Strings)

```
string a = "Hello ";  
string b = "World";  
string c = a + b;  
cout << c;
```

Output:

Hello World

◆ 2. Length of String

```
string text = "Programming";  
cout << text.length();
```

◆ 3. Accessing Characters

```
string word = "Example";  
cout << word[0]; // First character
```

◆ 4. Comparing Strings

```
string s1 = "Apple";  
string s2 = "Banana";
```

```
if(s1 == s2)  
    cout << "Equal";  
else  
    cout << "Not Equal";
```

◆ 5. Substring

```
string str = "Computer";  
cout << str.substr(0, 4);
```

Output:

Comp

◆ 6. Finding a Word

```
string str = "Welcome to C++";  
cout << str.find("C++");
```

◆ 7. Taking Input with Spaces

```
string name;  
getline(cin, name);
```

2 String Functions Using C-Style Strings (<cstring>)

Include:

```
#include <cstring>
```

Function	Purpose
strlen(str)	Returns length
strcpy(a,b)	Copies string
strcat(a,b)	Concatenates
strcmp(a,b)	Compares

Example:

```
#include <iostream>

#include <cstring>

using namespace std;


int main() {

    char str1[20] = "Hello";

    char str2[20] = "World";


    strcat(str1, str2);

    cout << str1;


    return 0;

}
```

Difference Between string and C-Style Strings

Feature	string Class	C-Style String
Easy to use	Yes	Moderate
Safe	More safe	Less safe
Operators	Supports +, ==	Uses functions
Recommended	Yes	Old method

6. Introduction to Object-Oriented Programming

Q1. Explain the key concepts of Object-Oriented Programming (OOP).

→

Object-Oriented Programming (OOP) is a programming paradigm based on the concept of **objects** and **classes**. It helps in organizing code in a structured and reusable way.

1 Class

A **class** is a blueprint or template used to create objects.

It contains:

- Data members (variables)
- Member functions (methods)

Example:

```
class Student {  
public:  
    string name;  
    int age;  
  
    void display() {  
        cout << name << " " << age;  
    }  
};
```

2 Object

An **object** is an instance of a class.
It represents a real-world entity.

Example:

```
Student s1;  
s1.name = "Ali";  
s1.age = 20;  
s1.display();
```

3 Encapsulation

Encapsulation means **wrapping data and functions together into a single unit (class)** and restricting direct access using access specifiers.

Access specifiers:

- private
- public
- protected

Example:

```
class Account {  
private:  
    double balance;  
public:  
    void setBalance(double b) {  
        balance = b;  
    }  
  
    double getBalance() {  
        return balance;  
    }  
};
```

4 Abstraction

Abstraction means **showing only essential details and hiding internal implementation**.

It is achieved using:

- Classes
- Access specifiers
- Functions

Example: User can call deposit() without knowing internal logic.

5 Inheritance

Inheritance allows one class to **acquire properties of another class**.

It promotes code reuse.

Example:

```
class Animal {  
public:  
    void eat() {  
        cout << "Eating";  
    }  
};  
  
class Dog : public Animal {  
};
```

Here, Dog inherits eat() from Animal.

Polymorphism

Polymorphism means **one function behaving in different ways**.

Two types:

- Compile-time (Function Overloading)
- Run-time (Function Overriding)

Example (Function Overloading):

```
int add(int a, int b) {  
    return a + b;  
}
```

```
double add(double a, double b) {  
    return a + b;  
}
```

Summary of OOP Concepts

Concept	Meaning
Class	Blueprint of objects
Object	Instance of class
Encapsulation	Wrapping data & methods
Abstraction	Hiding internal details
Inheritance	Acquiring properties
Polymorphism	Many forms of one function

Q2. What are classes and objects in C++? Provide an example.



◆ Class

A **class** is a blueprint or template used to create objects.
It defines:

- Data members (variables)
- Member functions (methods)

A class does not occupy memory until an object is created.

Syntax:

```
class ClassName {  
    access_specifier:  
        data_members;  
        member_functions();  
};
```

◆ Object

An **object** is an instance of a class.

It represents a real-world entity and occupies memory.

Objects are used to access the data and functions of a class.

Example Program

```
#include <iostream>  
  
using namespace std;  
  
class Student {  
public:
```

```
string name;

int age;

void display() {
    cout << "Name: " << name << endl;
    cout << "Age: " << age << endl;
}

};

int main() {
    Student s1;    // Object creation

    s1.name = "Rahul";
    s1.age = 20;

    s1.display();  // Calling member function

    return 0;
}
```

Explanation

- Student is a class.
- s1 is an object of the class.
- Data members: name, age
- Member function: display()
- The object is used to access class members using the dot (.) operator.

Q3. What is inheritance in C++? Explain with an example.



Inheritance is a feature of Object-Oriented Programming (OOP) in which one class acquires the properties and behavior of another class.

- The existing class is called the **Base (Parent) class**.
- The new class is called the **Derived (Child) class**.

Inheritance helps in:

- Code reusability
- Reducing duplication
- Creating hierarchical relationships

Syntax of Inheritance

```
class DerivedClass : access_specifier BaseClass {  
    // body  
};
```

Example Program

```
#include <iostream>  
  
using namespace std;  
  
// Base class  
class Animal {  
public:  
    void eat() {  
        cout << "Animal is eating" << endl;  
    }  
};
```

```
// Derived class
class Dog : public Animal {
public:
    void bark() {
        cout << "Dog is barking" << endl;
    }
};

int main() {
    Dog d;
    d.eat(); // Inherited function
    d.bark(); // Own function

    return 0;
}
```

Explanation

- Animal is the base class.
- Dog is the derived class.
- Dog inherits the function eat() from Animal.
- The object d can access both eat() and bark().

Types of Inheritance in C++

1. **Single Inheritance** – One base, one derived class
2. **Multiple Inheritance** – One derived class from multiple base classes
3. **Multilevel Inheritance** – Derived class becomes base for another class
4. **Hierarchical Inheritance** – Multiple derived classes from one base class
5. **Hybrid Inheritance** – Combination of two or more types

Q4. What is encapsulation in C++? How is it achieved in classes?



Encapsulation is the process of **wrapping data (variables) and functions (methods) together into a single unit**, which is a class.

It also means **restricting direct access to data** and allowing access only through public methods.

◆ How is Encapsulation Achieved in C++?

Encapsulation is achieved using:

1. **Classes**
2. **Access specifiers**
 - private
 - public
 - protected

Key Idea:

- Data members are usually declared as **private**
- Public functions are used to access or modify the data

Example Program

```
#include <iostream>
using namespace std;

class BankAccount {
private:
    double balance; // Hidden data
```

```

public:
    void setBalance(double b) {
        balance = b;
    }

    double getBalance() {
        return balance;
    }
};

int main() {
    BankAccount acc;

    acc.setBalance(5000);
    cout << "Balance: " << acc.getBalance();

    return 0;
}

```

Explanation

- balance is declared as **private**, so it cannot be accessed directly.
- It can only be accessed using setBalance() and getBalance().
- This protects the data from unauthorized access.

✗ Not allowed:

```
acc.balance = 5000; // Error (private member)
```

Advantages of Encapsulation

- Data security
- Control over data
- Improves code maintainability
- Prevents accidental modification